

10 — Production Fine-Tuning: End-to-End Pipeline

Time: ~5-6 hours | **Level:** Professional

What you'll learn:

- Complete walkthrough of our Mental Health LLM fine-tuning project
- How every concept from Notebooks 01-09 comes together
- Data pipeline: loading, validation, prompt engineering
- Training pipeline: QLoRA setup, Trainer, evaluation
- Serving pipeline: FastAPI, model loading, inference
- Production concerns: testing, Docker, monitoring, CI/CD

Prerequisites: All previous notebooks (01-09)

This notebook maps concepts to code

Every section shows WHERE in our project each concept from the learning path is applied.

Notebook 01 (ML Foundations)	→ src/data/__init__.py (pandas, scaling)
Notebook 02 (Classical ML)	→ src/model/evaluator.py
(metrics, train/test split)	
Notebook 03 (Neural Networks)	→ Understanding what the model IS
Notebook 04 (PyTorch)	→ src/model/__init__.py
(training loop, optimiser)	
Notebook 05 (NLP)	→ src/data/prompt_formatter.py
(tokenisation)	
Notebook 06 (RNNs)	→ Why we chose Transformers
over RNNs	
Notebook 07 (Transformers)	→ The architecture of Phi-3.5-mini
Notebook 08 (HuggingFace)	→ AutoModelForCausalLM,
Trainer, tokenizer	
Notebook 09 (Fine-Tuning)	→ QLoRA config, LoRA targets,
merge	

```
In [ ]: import os
import sys
import yaml
import pandas as pd
import numpy as np
```

```
# Add project root to path
project_root = os.path.abspath('.')
if project_root not in sys.path:
    sys.path.insert(0, project_root)

print(f'Project root: {project_root}')
print(f'\nProject structure:')
for item in sorted(os.listdir(project_root)):
    if not item.startswith('.') and item != '__pycache__':
        print(f' {item}/')
```

Stage 1: Configuration (YAML + Pydantic)

File: configs/training_config.yaml + src/utils/config.py

Concept: All hyperparameters in one place, validated at load time.

Why: Hardcoded values = silent bugs. Config files = reproducible experiments.

```
In [ ]: # — Load and inspect the training config —————

config_path = os.path.join(project_root, 'configs', 'training_config.yaml')

if os.path.exists(config_path):
    with open(config_path) as f:
        config = yaml.safe_load(f)

    print('=== Training Configuration ===')
    for section, values in config.items():
        print(f'\n[{section}]')
        if isinstance(values, dict):
            for k, v in values.items():
                print(f' {k}: {v}')
        else:
            print(f' {values}')
else:
    print(f'Config not found at {config_path}')
    print('This is expected if you haven\'t set up the project structure yet')
    # Show what the config looks like
    example_config = {
        'model': {
            'name': 'microsoft/Phi-3.5-mini-instruct',
            'max_seq_length': 512,
        },
        'quantisation': {
            'load_in_4bit': True,
            'bnb_4bit_quant_type': 'nf4',
            'bnb_4bit_compute_dtype': 'bfloat16',
        },
        'lora': {
            'r': 16,
            'alpha': 32,
            'dropout': 0.05,
```

```

        'target_modules': ['q_proj', 'k_proj', 'v_proj', 'o_proj',
                           'gate_proj', 'up_proj', 'down_proj'],
    },
    'training': {
        'batch_size': 2,
        'gradient_accumulation_steps': 4,
        'epochs': 3,
        'learning_rate': 2e-4,
        'warmup_ratio': 0.1,
        'lr_scheduler_type': 'cosine',
        'eval_steps': 200,
        'early_stopping_patience': 3,
    },
}
print('\n=== Example Configuration ===')
print(yaml.dump(example_config, default_flow_style=False))

```

Stage 2: Data Loading & Validation

File: `src/data/__init__.py`

Concepts from Notebook 01: Pandas DataFrames, feature scaling, data validation.

Key design decisions:

1. Unicode normalisation (non-breaking hyphens in column names)
2. Schema validation (check all expected columns exist)
3. **No data leakage:** scaler fit on training set ONLY

```

In [ ]: # — Data loading (same as src/data/__init__.py) —————

data_path = os.path.join(project_root, 'mental_health_dataset.csv')

if os.path.exists(data_path):
    df = pd.read_csv(data_path)

    # Normalise column names (handles unicode non-breaking hyphens)
    df.columns = [c.replace('\u2011', '-').replace('\u00a0', ' ').strip() for c in df.columns]

    print(f'Dataset: {df.shape[0]} rows × {df.shape[1]} columns')
    print(f'\nColumns: {list(df.columns)}')
    print(f'\nSample row:')
    print(df.iloc[0].to_dict())

    # Data quality checks
    print(f'\n--- Data Quality ---')
    print(f'Missing values: {df.isnull().sum().sum()}')
    print(f'Duplicates: {df.duplicated().sum()}')

    # Score distributions
    score_cols = [c for c in df.columns if 'Score' in c]
    if score_cols:
        print(f'\n--- Score Statistics ---')

```

```

        print(df[score_cols].describe().round(3))
    else:
        print('Dataset not found. Run from the project root directory.')

```

Stage 3: Prompt Engineering

File: `src/data/prompt_formatter.py`

Concepts from Notebooks 05 & 08: Tokenisation, chat templates.

This is where we convert tabular patient data into text the LLM can understand.

CSV Row → Structured Prompt (user message) → Model Response (assistant message)

```

In [ ]: # — Prompt formatting (simplified from src/data/prompt_formatter.py) —

def build_prompt(row):
    """Convert a CSV row to a Phi-3 user prompt."""
    parts = [
        '<|user|>',
        'You are a clinical mental health assessment assistant.',
        'Assess the following patient:',
        '',
    ]

    # Build patient profile from available columns
    for col in ['Age', 'Gender', 'Symptoms', 'Therapy History', 'Medication']:
        if col in row and pd.notna(row[col]):
            parts.append(f'- {col}: {row[col]}')

    # Add scores
    for col in ['Depression Score', 'Anxiety Score', 'General Well-Being Score']:
        if col in row and pd.notna(row[col]):
            parts.append(f'- {col}: {row[col]:.2f}')

    parts.extend(['', '<|end|>', '<|assistant|>'])
    return '\n'.join(parts)

def build_completion(row):
    """Build the target completion (what the model should generate)."""
    dep = row.get('Depression Score', 0)
    anx = row.get('Anxiety Score', 0)
    avg = (dep + anx) / 2

    if avg > 0.6:
        severity = 'High'
    elif avg > 0.3:
        severity = 'Moderate'
    else:
        severity = 'Low'

    # In the real code, we use hash-seeded templates for diversity

```

```

        narrative = (f'This patient presents with {severity.lower()}-risk indica
                    f'Depression score ({dep:.2f}) and anxiety score ({anx:.2f})
                    f'suggest {"immediate" if severity == "High" else "ongoing"}

        return f'Severity: {severity}\n\n{narrative}\n<|end|>'

if os.path.exists(data_path):
    row = df.iloc[0]
    prompt = build_prompt(row)
    completion = build_completion(row)

    print('=== Full Training Example ===')
    print(prompt)
    print(completion)
    print()
    print('💡 The model learns to generate the completion given the prompt.')
    print('    Prompt tokens are masked (loss = -100) during training.')

```

Stage 4: Model Setup (QLoRA)

File: `src/model/__init__.py`

Concepts from Notebooks 07 & 09: Transformer architecture, LoRA, quantisation.

This stage:

1. Loads Phi-3.5-mini in 4-bit (NF4)
2. Applies LoRA adapters to attention + FFN layers
3. Sets up the data collator (prompt masking)

```

In [ ]: # — Model loading (conceptual – needs GPU to run) —————

print('=== What src/model/__init__.py does ===')
print()
print('1. load_tokenizer(model_name)')
print('    → AutoTokenizer.from_pretrained("microsoft/Phi-3.5-mini-instruct")')
print('    → Sets pad_token = eos_token (Phi-3 has no pad token by default)')
print()
print('2. load_base_model(model_name, quantisation_config)')
print('    → BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4")')
print('    → AutoModelForCausalLM.from_pretrained(..., quantization_config=...)')
print('    → ~2GB VRAM instead of 15.2GB')
print()
print('3. apply_lora(model, lora_config)')
print('    → LoraConfig(r=16, alpha=32, target_modules=[q,k,v,o,gate,up,down])')
print('    → get_peft_model(model, lora_config)')
print('    → Only 0.58% of parameters are trainable')
print()
print('4. PromptCompletionCollator(tokenizer, max_len)')
print('    → Tokenises prompt + completion')
print('    → Sets labels = -100 for prompt tokens (model doesn\'t learn to pr')
print('    → Sets labels = -100 for padding tokens')
print('    → Only completion tokens contribute to the loss')

```

```
print()
print('💡 Step 4 is crucial: without prompt masking, the model wastes capacity  
print('    learning to predict the prompt (which is always the same format).'
```

Stage 5: Training

File: `scripts/train.py` + `src/model/__init__.py`

Concepts from Notebooks 03-04: Training loop, gradient descent, optimisers.

We use HuggingFace `Trainer` which handles:

- Mixed precision training (fp16/bf16)
- Gradient accumulation (effective batch size = $2 \times 4 = 8$)
- Learning rate scheduling (cosine warmup)
- Early stopping (patience = 3)
- Logging and checkpointing

```
In [ ]: # — Training configuration walkthrough —————

print('=== Training Hyperparameters ===')
print()
training_params = {
    'per_device_train_batch_size': ('2', 'Small due to GPU memory constraint'),
    'gradient_accumulation_steps': ('4', 'Effective batch =  $2 \times 4 = 8$ '),
    'num_train_epochs': ('3', 'Usually 1-5 for fine-tuning (not 100+ like for pre-training)'),
    'learning_rate': ('2e-4', 'Typical for LoRA fine-tuning (10x higher than pre-training)'),
    'warmup_ratio': ('0.1', 'Gradually increase LR for first 10% of training steps'),
    'lr_scheduler_type': ('cosine', 'Smoothly decay LR to near-zero (see Notebooks 03-04)'),
    'fp16': ('True', 'Half-precision training (2x faster, less VRAM)'),
    'eval_strategy': ('steps', 'Evaluate during training, not just at end'),
    'eval_steps': ('200', 'Evaluate every 200 training steps'),
    'save_strategy': ('steps', 'Save checkpoints periodically'),
    'load_best_model_at_end': ('True', 'Use best checkpoint, not last'),
    'metric_for_best_model': ('eval_loss', 'Select best model by validation loss'),
    'early_stopping_patience': ('3', 'Stop if no improvement for 3 evaluations')
}

for param, (value, explanation) in training_params.items():
    print(f' {param:35s} = {value:8s} # {explanation}')

print(f'\n--- Training Math ---')
n_samples = 8000 # 80% of 10K
batch_size = 8
steps_per_epoch = n_samples // batch_size
total_steps = steps_per_epoch * 3

print(f'Training samples: {n_samples}')
print(f'Effective batch size: {batch_size}')
print(f'Steps per epoch: {steps_per_epoch}')
print(f'Total steps: {total_steps}')
```

```
print(f'Warmup steps: {int(total_steps * 0.1)}')
print(f'Evaluations: {total_steps // 200}')
```

Stage 6: Evaluation

File: `src/model/evaluator.py` + `scripts/evaluate.py`

Concepts from Notebook 02: Classification metrics, confusion matrix.

We evaluate two tasks simultaneously:

1. **Classification:** extract severity label → F1, accuracy, confusion matrix
2. **Generation quality:** extract narrative → ROUGE, BERTScore

```
In [ ]: # — Evaluation metrics explained —————
import re

def extract_severity(text):
    """Extract severity label from model output."""
    match = re.search(r'Severity:\s*(High|Moderate|Low)', text, re.IGNORECASE)
    return match.group(1) if match else 'Unknown'

# Simulated evaluation
print('=== Evaluation Pipeline ===')
print()
print('1. Generate predictions for each test sample:')
print('    model.generate(prompt) → "Severity: High\n\nBased on clinical in')
print()
print('2. Extract structured fields:')
print('    extract_severity(output) → "High"')
print('    extract_summary(output) → "Based on clinical indicators..."')
print()
print('3. Classification metrics (Notebook 02):')
print('    - Accuracy: overall correct predictions')
print('    - F1 (macro): balanced score across High/Moderate/Low')
print('    - Confusion matrix: see which classes get confused')
print()
print('4. Generation metrics:')
print('    - ROUGE-1/2/L: n-gram overlap with reference (Notebook 05)')
print('    - BERTScore: semantic similarity using BERT embeddings')
print()
print('5. Output:')
print('    - metrics.json: aggregate numbers')
print('    - per_sample_results.csv: every prediction for error analysis')
print()

# Demo severity extraction
outputs = [
    'Severity: High\n\nPatient presents with elevated risk indicators.',
    'Severity: Moderate\n\nClinical assessment suggests moderate concerns.',
    'The patient seems okay.', # malformed – model didn't follow format
]
```

```

for out in outputs:
    sev = extract_severity(out)
    print(f'  "{out[:50]}..." → Severity: {sev}')

```

Stage 7: Serving

File: `src/serving/app.py` + `src/serving/schemas.py` + `scripts/serve.py`

Production concerns:

- FastAPI for high-performance API
- Pydantic validation on inputs (age 1-120, scores 0-1)
- Health check endpoint for container orchestration
- Request-ID middleware for tracing
- Structured JSON logging

```

In [ ]: # — API schema (from src/serving/schemas.py) —————
from dataclasses import dataclass

print('=== API Design ===')
print()
print('POST /assess')
print('Request body:')
request_example = {
    'age': 34,
    'gender': 'Female',
    'symptoms': 'Persistent low mood, fatigue, loss of interest',
    'therapy_history': '6 months CBT',
    'medication': 'None',
    'depression_score': 0.72,
    'anxiety_score': 0.45,
    'wellbeing_score': 0.35,
}
for k, v in request_example.items():
    print(f'  {k}: {v}')

print('\nResponse:')
response_example = {
    'severity': 'High',
    'assessment': 'Based on clinical indicators, this patient presents with.',
    'model_version': 'phi3.5-mental-health-v1',
    'confidence': 0.87,
}
for k, v in response_example.items():
    print(f'  {k}: {v}')

print('\nGET /health')
print('  → {"status": "healthy", "model_loaded": true}')

print('\n=== Validation (Pydantic) ===')
print('  age: 1-120 (rejects negative or unreasonable values)')

```



```
print('  scores: 0.0-1.0 (rejects out-of-range)')
print('  All string fields: stripped and checked for non-empty')
```

Stage 8: Deployment

Files: Dockerfile, docker-compose.yml, Makefile

```
make docker-build    # Build containers
make docker-serve    # Run API server
make test            # Run test suite
make lint            # Code quality checks
```

Docker multi-stage build:

```
Base image (Python + CUDA + deps)
├─ Train target: + training scripts + data
└─ Serve target: + serving code only (lean, ~2GB smaller)
```

Production checklist:

- ☒ Pydantic input validation
- ☒ Health check endpoint
- ☒ Request-ID middleware
- ☒ Structured JSON logging
- ☒ Docker multi-stage build
- ☒ docker-compose with GPU passthrough
- ☒ Unit tests (data, prompts, API, evaluator)
- ☒ Early stopping to prevent overfitting
- ☒ Config files (no hardcoded values)
- ☒ .env for secrets (HF_TOKEN, WANDB_KEY)

Stage 9: The Full Pipeline Diagram

```
In [ ]: # — Pipeline visualisation —————

pipeline_stages = [
    ('1. Config',          'configs/*.yaml',          'YAML + Pydantic valid'),
    ('2. Data Load',       'src/data/__init__.py',     'Pandas, schema valid'),
    ('3. Formatting',      'src/data/prompt_formatter.py', 'Phi-3 chat template'),
    ('4. Model Setup',     'src/model/__init__.py',     'QLoRA: 4-bit model +'),
    ('5. Training',        'scripts/train.py',         'Trainer + early stop'),
    ('6. Evaluation',      'src/model/evaluator.py',   'ROUGE + BERTScore +'),
    ('7. Merge',           'src/model/__init__.py',     'LoRA weights merged'),
    ('8. Serving',         'src/serving/app.py',       'FastAPI + Pydantic +'),
    ('9. Deploy',          'Dockerfile + docker-compose', 'Multi-stage Docker +'),
    ('10. Monitor',        'src/utils/logger.py',      'Structured JSON logg')
]
```

```

print('
print('
print('
print()

for stage, file, description in pipeline_stages:
    print(f' {stage:15s} | {file:30s} | {description}')
    if stage != '10. Monitor':
        print(f' {stage:15s} | {file:30s} | ')

print()
print('💡 Each stage is independently testable and configurable.')
print('  Change the model? Update configs/training_config.yaml.')
print('  Change the data? Update src/data/.')
print('  Change the API? Update src/serving/.')

```

✓ Learning Path Complete!

What you've learned across 10 notebooks:

#	Topic	Key Concept
01	ML Foundations	NumPy, Pandas, statistics, feature engineering
02	Classical ML	Regression, classification, metrics, bias-variance
03	Neural Networks	Forward pass, loss, backprop, gradient descent
04	PyTorch	Tensors, autograd, nn.Module, training loops
05	NLP	Tokenisation, BoW, TF-IDF, embeddings, BPE
06	RNNs	Sequential memory, vanishing gradients, LSTM
07	Transformers	Attention, multi-head, positional encoding
08	HuggingFace	Pre-trained models, tokenisers, pipelines
09	Fine-Tuning	LoRA, QLoRA, PEFT, RLHF
10	Production	End-to-end pipeline: config → data → train → serve

Your next steps:

1. **Run the project:** `make train` → `make evaluate` → `make serve`
2. **Experiment:** change rank, learning rate, target modules in the config
3. **Extend:** add DPO alignment, swap the base model, add RAG
4. **Deploy:** push to cloud (AWS, GCP, Azure) with proper CI/CD

Built as a learning resource for the Mental Health LLM Fine-Tuning project. From zero to production — one notebook at a time.